Al-Farabi Kazakh National University
Department of Computer
Algorithm and Data Structure
Final Project: Movie Recommendation Engin
Professor: ~Irina Ualiyeva
Prepared By: SHARIFI MOHAMMAD HADI

Movie Recommendation System

1. Introduction

The Movie Recommendation System is a console-based application developed as part of the Data Structures and Algorithms course. The primary objective of this system is to model a simplified yet structurally accurate representation of how large-scale recommendation engines operate in real-world platforms such as Netflix, YouTube, and Amazon Prime Video.

The system is designed to process a collection of movies and provide personalized recommendations based on user interactions, including viewing history and rating behavior. In order to achieve computational efficiency and scalability, the system is implemented using fundamental data structures such as hash tables, queues, and graph-based representations combined with graph traversal algorithms.

This project emphasizes not only functional correctness but also algorithmic efficiency, modular software design, and adherence to theoretical principles of data structures.

2. Motivation and Purpose

The motivation behind this project originates from the increasing importance of recommendation systems in modern digital platforms. These systems rely heavily on efficient data organization and graph-based relationships to process large volumes of user data in real time.

The purpose of this implementation is to bridge theoretical knowledge with practical application by:

1. Demonstrating the use of hash tables for constant-time data retrieval and updates
2. Applying queue data structures for managing temporal user interaction history
3. Utilizing graph structures to model relationships between entities
4. Implementing Breadth-First Search (BFS) for systematic exploration of related nodes

Furthermore, the project aims to strengthen understanding of algorithmic efficiency and real-world system design patterns.

3. Data Structures and Their Justification

3.1 Hash Tables (Dictionaries)

The system utilizes two primary hash tables implemented using Python dictionaries:

Movie Registry

The Movie Registry serves as the core data repository, where each movie is uniquely identified by a movie ID. Each entry contains structured metadata including the title, genre set, cumulative rating sum, and total number of ratings.

This design ensures:

1. Constant-time complexity O(1) for retrieval operations
2. Efficient update operations for rating modifications
3. Elimination of linear search overhead

User Ratings Dictionary

A secondary hash table is maintained to store user-specific ratings. This structure maps movie identifiers to user-assigned rating values, enabling the system to:

1. Prevent duplicate ratings
2. Efficiently update previously assigned ratings
3. Maintain consistency in aggregated rating computation

3.2 Queue (FIFO – Viewing History)

A First-In-First-Out (FIFO) queue is implemented using Python's deque structure with a fixed maximum capacity of five elements.

This structure ensures that:

1. The most recently watched movies are retained
2. The oldest entries are automatically removed upon exceeding capacity
3. Temporal ordering of user activity is preserved

This mechanism is essential for modeling short-term user behavior, which directly influences recommendation generation.

3.3 Graph Representation (Adjacency List)

The recommendation engine is modeled using a graph data structure, where:

1. Each movie represents a vertex (node)
2. An edge exists between two movies if they share at least one common genre

The graph is implemented using an adjacency list representation in the form of a dictionary of sets.

This approach is chosen due to:

1. Memory efficiency compared to adjacency matrices
2. Fast adjacency lookup operations

3. Suitability for sparse graph structures

3.4 Breadth-First Search (BFS)

Breadth-First Search is employed as the primary graph traversal algorithm for generating recommendations.

BFS is selected because it systematically explores nodes level by level, ensuring that closely related movies (based on genre similarity) are prioritized in the recommendation output.

The algorithm:

1. Initiates traversal from a selected movie node
2. Visits all directly connected neighbors
3. Expands exploration iteratively across levels
4. Excludes previously watched movies from results
5. Produces a ranked list of recommended content

4. Dataset Description

The system operates on a structured dataset consisting of twenty manually defined movies. Each movie record includes:

1. Title
2. Genre classification
3. Aggregated rating sum
4. Total number of ratings

The dataset structure is conceptually aligned with real-world datasets such as MovieLens, which are widely used in academic research and industrial recommendation systems.

5. System Architecture and Operational Logic

The system operates according to the following logical pipeline:

1. The user selects a movie from the dataset
2. The selected movie is recorded in the viewing history queue
3. The user assigns a rating to the movie
4. The rating is processed using an aggregated update mechanism
5. The graph structure identifies relationships based on genre similarity
6. BFS traversal is executed to explore related movies
7. The recommendation list is generated and sorted by average rating

This pipeline ensures consistency, efficiency, and logical coherence between user interaction and system response.

6. Rating System and Complexity Analysis

The rating system is designed using an aggregated storage model in which only the sum of ratings and the count of ratings are maintained.

The average rating is computed using the formula:

Average Rating = (Sum of Ratings) / (Number of Ratings)

Update Mechanism:

1. For a first-time rating, the count is incremented
2. For rating updates, the previous value is subtracted and replaced with the new value

This approach guarantees:

1. Constant time complexity O(1) for updates
2. Avoidance of storing individual rating records
3. Efficient memory utilization

7. Time Complexity Analysis

Operation	Data Structure	Time Complexity	Explanation
Movie lookup	Hash Table	O(1)	Direct key-based access
Rating update	Hash Table	O(1)	Constant-time arithmetic update
Viewing history update	Queue (deque)	O(1)	FIFO insertion and deletion
Graph construction	Graph	$O(n^2)$	Pairwise comparison of movies
BFS traversal	Graph	O(V + E)	Visits all vertices and edges

Comparison with Linear Structures

If a list-based implementation were used instead of hash tables, the complexity of lookup and update operations would increase to O(n), resulting in significantly reduced performance for large datasets.

8. Code Quality and Design Principles

The implementation follows structured programming principles with modular function decomposition. Each functional component is isolated into independent functions, improving readability, maintainability, and scalability.

The code adheres to PEP8 conventions, ensuring:

- Consistent formatting
- Clear naming conventions

- Logical separation of concerns

9. Graph and Algorithmic Justification

The graph is constructed using an adjacency list representation to efficiently model relationships between movies based on shared genres.

Breadth-First Search is utilized due to its ability to:

- Explore nearest relationships first
- Maintain systematic traversal order
- Ensure relevance in recommendation output

This combination provides an effective balance between computational efficiency and recommendation quality.

10. Conclusion

The Movie Recommendation System successfully demonstrates the practical implementation of fundamental data structures and algorithms within a real-world inspired application.

By integrating hash tables, queues, and graph-based traversal algorithms, the system achieves efficient data processing, structured relationship modeling, and dynamic recommendation generation.

This project reinforces core concepts of algorithmic design, computational complexity, and system architecture, thereby bridging theoretical knowledge with practical software engineering applications.